ARQUITETURA DE SOFTWARE • C4 MODEL

Smart Fly — Planejador de Passagens Aéreas

Documentação arquitetural completa em 4 níveis (Contexto, Contêineres, Componentes, Modelo de Dados e Fluxos de Sequência) da plataforma full-stack para planejamento anual de voos, cotações, gestão financeira e alertas inteligentes.

Repositório: Taiolor/flight_planner

Stack: React 19 + Vite 8 + Express 4 + tRPC 11 + Drizzle ORM + MySQL

Rota Principal: GRU / CGH ⇔ NVT

1. Visão Geral do Sistema

O **Smart Fly** é uma aplicação web progressiva (PWA) full-stack voltada para o planejamento anual das 52 semanas de voo, cotação automatizada e manual de passagens aéreas, controle de emissão de bilhetes (em R$ e milhas Smiles / LATAM Pass), gestão financeira com projeções sazonais e envio automatizado de alertas (Web Push e E-mail).

CAMADA	TECNOLOGIAS	MÓDULOS PRINCIPAIS
Frontend (SPA / PWA)	React 19, Vite 8, TypeScript, Tailwind CSS 4, Wouter, TanStack Query, tRPC Client, Recharts, Radix UI / shadcn, jsPDF	`client/src/App.tsx`, `Home.tsx`, `CalendarView.tsx`, `FlightQuotes.tsx`, `FinancialDashboard.tsx`, `sw.js`
Backend (API & Jobs)	Node.js, Express 4, tRPC 11, Zod, Helmet (CSP), Express Rate Limit, Web-Push (VAPID), Resend SDK	`server/_core/index.ts`, `server/routers.ts`, `routers/quotes.ts`, `routers/financial.ts`, `pushNotifications.ts`
Banco de Dados	MySQL 8 / TiDB, Drizzle ORM, Drizzle Kit (25+ migrations aplicadas)	`drizzle/schema.ts`, `server/db.ts`, `server/financialDb.ts`

2. Nível 1 — Diagrama de Contexto do Sistema (System Context)

O **Nível 1 (Contexto)** apresenta o **Smart Fly** como uma caixa central, destacando os perfis de usuários que interagem com a plataforma e os sistemas externos consumidos.

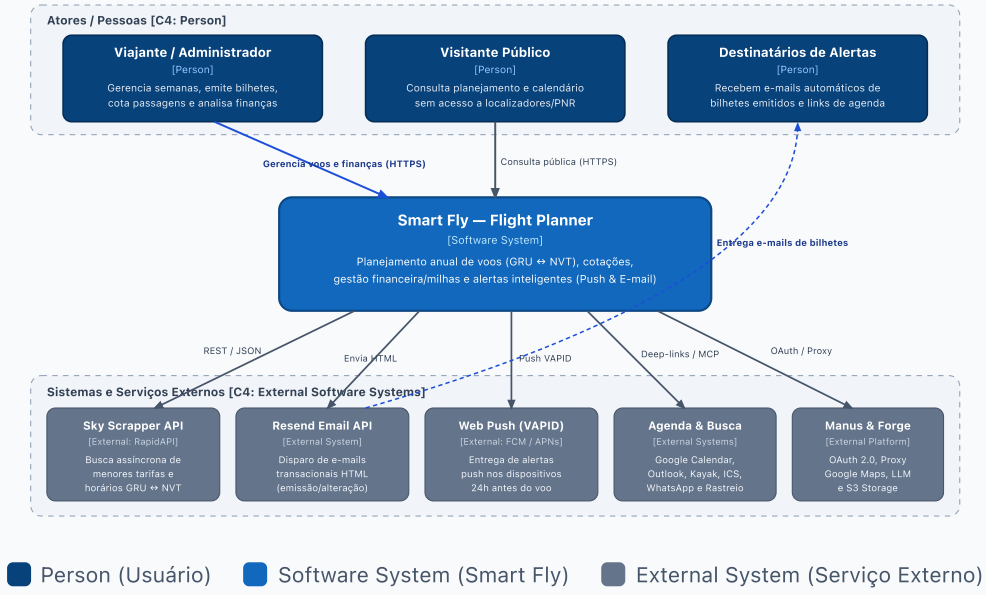


FIGURA 1 — DIAGRAMA C4 NÍVEL 1: CONTEXTO DO SISTEMA SMART FLY

3. Nível 2 — Diagrama de Contêineres (Container Diagram)

O **Nível 2 (Contêineres)** amplia a fronteira do sistema Smart Fly para mostrar suas unidades de execução independentes (SPA React, Service Worker PWA, Servidor Node/Express/tRPC, Agendador Horário em Background e Banco de Dados MySQL/TiDB).

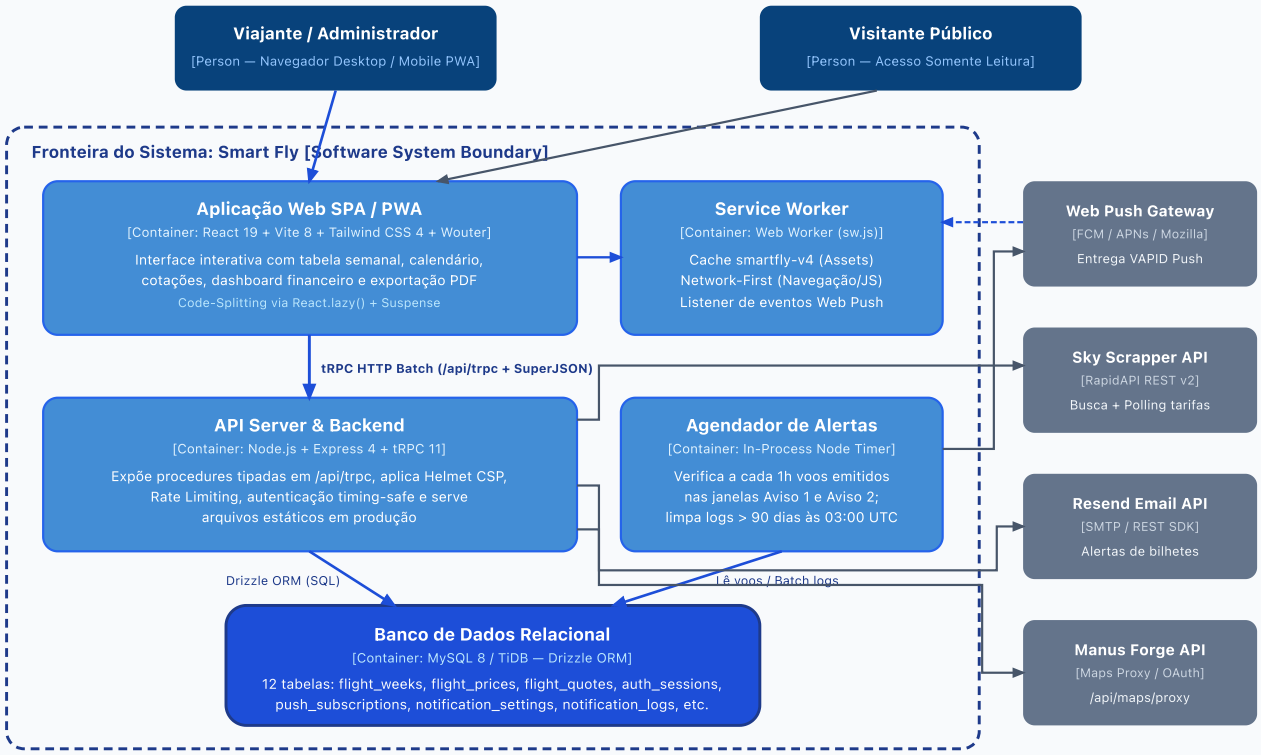


FIGURA 2 — DIAGRAMA C4 NÍVEL 2: CONTÊINERES DO SISTEMA SMART FLY

Responsabilidades dos Contêineres

- **Aplicação Web SPA / PWA (`client/`):** SPA em React 19 + Vite 8 com divisão de código (`React.lazy`) para carregamento sob demanda das rotas secundárias e chunks dedicados (`vendor-react`, `vendor-ui`, `vendor-trpc`, `vendor-charts`, `vendor-pdf`).
- **Service Worker (`client/public/sw.js`):** Implementa cache `smartfly-v4` (Cache-First para ícones/manifesto e Network-First para rotas e código) e processa eventos `push` e `notificationclick` em background.
- **API Server & Backend (`server/_core/index.ts`):** Servidor Express 4 protegido por Helmet (CSP estrito), Rate Limiting em duas camadas (200 req/15m geral; 20 req/15m em autenticação), proxy de mapas e roteador tRPC 11.
- **Agendador em Background (`server/pushNotifications.ts`):** Job iniciado por `startFlightNotificationJob()` que verifica a cada hora os voos emitidos próximos da janela de embarque e remove logs com mais de 90 dias diariamente às 03:00 UTC.

4. Nível 3 — Diagrama de Componentes (Component Diagram)

4.1. Componentes do Frontend (`client/src/`)

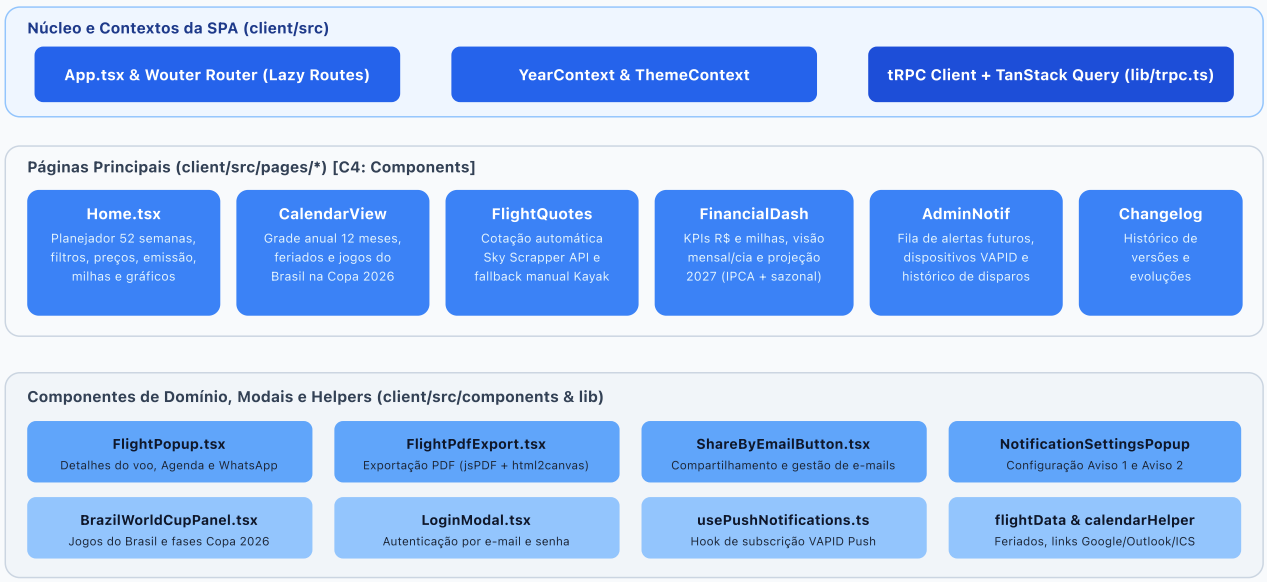


FIGURA 3 — DIAGRAMA C4 NÍVEL 3.1: COMPONENTES DO FRONTEND (REACT 19 SPA)

4.2. Componentes do Backend (`server/`)



FIGURA 4 — DIAGRAMA C4 NÍVEL 3.2: COMPONENTES DO BACKEND (EXPRESS + TRPC)

5. Nível 4 — Modelo de Dados (ER) e Fluxos de Sequência

O esquema relacional em `drizzle/schema.ts` é composto por **12 tabelas** no MySQL 8 / TiDB, ilustradas no Diagrama Entidade-Relacionamento abaixo:

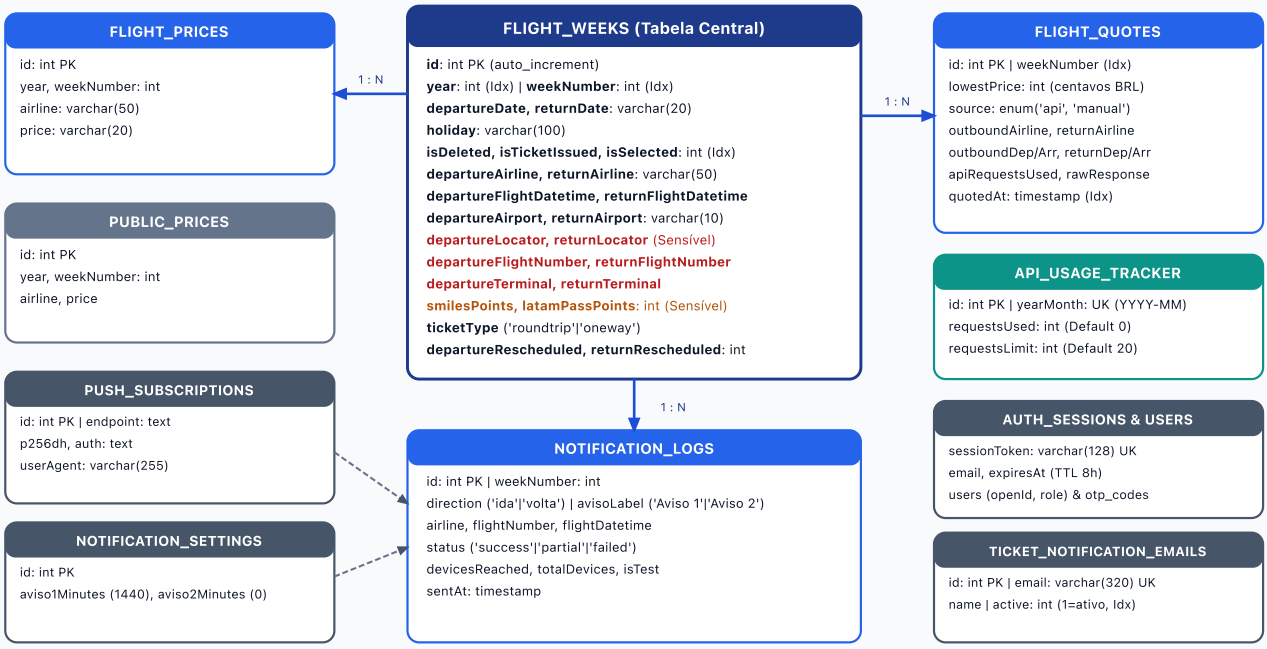


FIGURA 5 — DIAGRAMA C4 NÍVEL 4.1: MODELO ENTIDADE-RELACIONAMENTO (DRIZZLE/SCHEMA.TS)

5.2. Fluxo Dinâmico 1: Autenticação Timing-Safe e Proteção de Dados Sensíveis

Implementado em `server/routers.ts` e `server/flightAuthMiddleware.ts`. Visitantes sem sessão recebem a lista de semanas com localizadores, números de voo, terminais e milhas ocultados (`null`). O login utiliza comparação em tempo constante (`SHA-256` + `crypto.timingSafeEqual`).

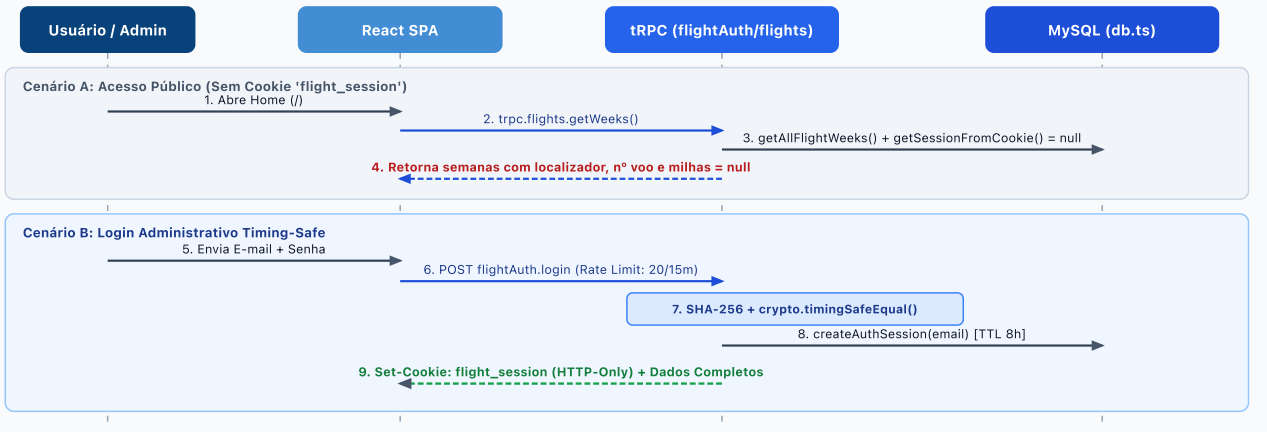


FIGURA 6 — DIAGRAMA DE SEQUÊNCIA: AUTENTICAÇÃO TIMING-SAFE E SANITIZAÇÃO DE DADOS PÚBLICOS

5.3. Fluxo Dinâmico 2: Emissão/Edição de Bilhete e Disparo Automático de E-mails

Ao atualizar uma semana via `flights.updateWeekStatus`, o servidor compara o estado anterior (`weekBefore`) com o novo estado para identificar se o bilhete foi recém-emitido, alterado ou excluído, enviando automaticamente um e-mail HTML (com links para Google Calendar, Outlook e Rastreio) via **Resend API** para os destinatários ativos.

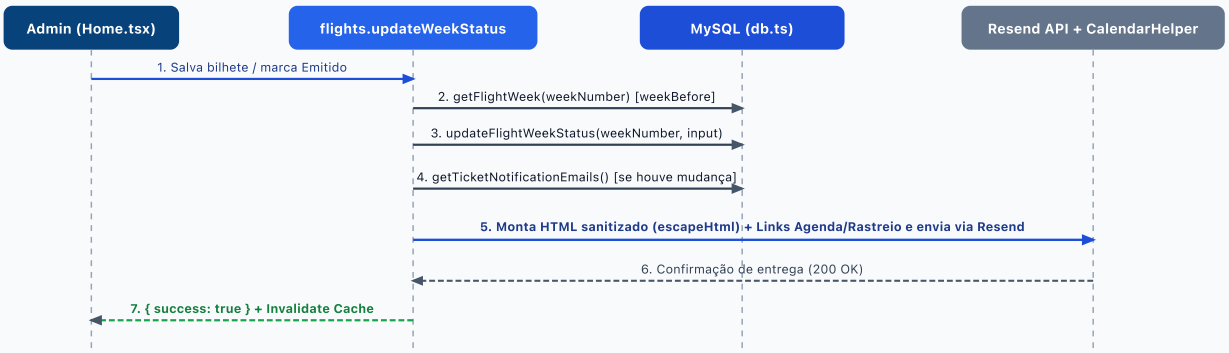


FIGURA 7 — DIAGRAMA DE SEQUÊNCIA: EMISSÃO/EDIÇÃO DE BILHETE E ALERTA POR E-MAIL

5.4. Fluxo Dinâmico 3: Cotação Sky Scrapper API (Polling Exponencial) & Job Web Push



FIGURA 8 — FLUXOS DE COTAÇÃO ASSÍNCRONA (SKY SCRAPPER) E AGENDADOR HORÁRIO DE ALERTAS PUSH (VAPID)

6. Matriz de Segurança (Sentinel) e Performance (Bolt)

MECANISMO ARQUITETURAL	IMPLEMENTAÇÃO NO PROJETO	ARQUIVO
Prevenção de Timing Attacks	Comparação de e-mail e senha usando hash fixo `SHA-256` (32 bytes) + `crypto.timingSafeEqual`.	`server/routers.ts`
Prevenção de IDOR Público	Rota pública `flights.getWeeks` omite localizadores, números de voo, terminais e milhas quando não há sessão ativa.	`server/routers.ts`
Sanitização Anti-XSS	Todos os campos interpolados em e-mails HTML passam por `escapeHtml()` antes do envio via Resend.	`emailNotification.ts`
Headers HTTP, CSP & Limites	`helmet` com `Content-Security-Policy` customizado, limite JSON de `2mb` e validação `.max(255)` no Zod.	`server/_core/index.ts`
Rate Limiting em 2 Camadas	`generalLimiter` (200 req/15m em `/api`) e `authLimiter` (20 req/15m em `/api/oauth` e `flightAuth.login`).	`trpcLoginRateLimit.ts`
Otimizações Bolt (DB & React)	Índices compostos no MySQL, `Promise.all` paralelo, batch insert de logs e memoização `O(1)` no frontend.	`pushNotifications.ts`, `financialDb.ts`